



Universidade do Minho
Escola de Ciências

Licenciatura em Ciência de Dados

Ambientes e Linguagens de Programação para Ciência de Dados

Grupo ALPCD_2

Docentes:

Pedro Manuel Rangel Santos Henriques

Tiago João Fernandes Baptista

Dean Kuijf (A106832)

Francisca Machado (A106863)

Gabriela Durães (A106858)

2024/2025

Índice

Introdução	4
Metodologia	5
Importação dos Dados	6
Recolha e Integração de Dados da API.....	6
Exportação de Dados	6
Função <i>export_to_csv1</i>	6
Função <i>export_to_csv2</i>	6
Tratamento de Dados	6
Funcionalidades	7
Trabalho Prático 1	7
Função <i>listar_trabalhos</i>	7
Função <i>full_time_jobs</i>	8
Função <i>remote_jobs</i>	8
Função <i>get_salary</i>	9
Função <i>skills</i>	9
Função <i>trabalhos_por_localidade</i>	10
Trabalho Prático 2	11
Função <i>informations</i>	11
Função <i>statistics</i>	12
Função <i>list_skills</i>	12
Função <i>informations2</i>	13
Conclusão	14

Índice de Figuras

Figura 1 - Importação da API	6
Figura 2 - Função export_to_csv1	6
Figura 3 - Função export_to_csv2	6
Figura 4 - Função limpar_html	6
Figura 5 - Função listar_trabalhos	7
Figura 6 - Exemplo output 'listar-trabalhos'	8
Figura 7 - Função full_time_jobs	8
Figura 8 - Exemplo output 'full-time-jobs'	8
Figura 9 - Função remote_jobs	8
Figura 10 - Exemplo output 'remote-jobs'	9
Figura 11 - Função get_salary	9
Figura 12 - Exemplo output 'get-salary'	9
Figura 13 - Função skills	9
Figura 14 - Exemplo output 'skills'	10
Figura 15 - Função trabalhos_por_localidade	10
Figura 16 - Output 'trabalhos-por-localidade'	10
Figura 17 - Exemplo pdf 'trabalhos-por-localidade'	10
Figura 18 - Função informations	11
Figura 19 - Função get_api_content	11
Figura 20 - Extração de dados HTML	11
Figura 21 - Função get_html_content	11
Figura 22 - Exemplo output 'informations'	12
Figura 23 - Função statistics	12
Figura 24 - Exemplo output 'statistics'	12
Figura 25 - Função list_skills	12
Figura 26 - Extração de dados HTML	12
Figura 27 - Exemplo output 'list-skills'	13
Figura 28 - Função informations2	13
Figura 29 - Exemplo output 'informations2'	13

Introdução

No âmbito da unidade curricular de Algoritmos e Linguagens de Programação para Ciência de Dados, foram propostos dois trabalhos práticos complementares que visam aplicar os conhecimentos adquiridos até ao momento.

Estes trabalhos têm como objetivo principal o desenvolvimento de uma ferramenta em Python, sob a forma de uma *Command Line Interface* (CLI), que permita interagir com a REST API e enriquecer os dados obtidos com informações recolhidas de websites externos. Para criar CLIs de forma estruturada e eficiente, utilizamos a biblioteca *typer*.

O TP1 focou-se no desenvolvimento de competências técnicas relacionadas com expressões regulares e manipulação de dados em Python, utilizando os módulos *re* e *requests*. A partir da API do itjobs.pt, foram implementadas funcionalidades para listar ofertas de emprego, filtrar resultados com base em critérios específicos, e exportar os dados em formato JSON e CSV.

O TP2 expandiu o projeto inicial, adicionando técnicas de *web scraping* com os módulos *beautifulsoup4* e *requests*. Este trabalho introduziu a recolha de informações de websites externos, como o AmbitionBox, com o objetivo de enriquecer os dados da API do itjobs.pt com informações detalhadas sobre as empresas, como a avaliação geral, a descrição da empresa e os principais benefícios. Além disso, foram implementadas funcionalidades para análise e contagem de vagas, bem como para listar as principais competências requisitadas para determinadas posições, organizando os resultados em formatos JSON ou CSV.

Estes projetos, desenvolvidos de forma incremental, têm como finalidade não apenas consolidar os conhecimentos técnicos adquiridos, mas também estimular a criatividade e a inovação através da exploração de fontes de dados alternativas e a integração de funcionalidades adicionais.

Metodologia

A metodologia adotada para o desenvolvimento deste projeto baseou-se numa abordagem iterativa e modular, com o objetivo de garantir a implementação eficiente e funcional das funcionalidades requisitadas. Para tal, utilizámos diversas bibliotecas do Python, tais como *typer*, *requests*, *re*, *csv*, *json* e *BeautifulSoup*, que permitiram manipulação de dados, integração com APIs e *Web Scraping*.

Foi desenvolvido um conjunto de funções para aceder e processar dados da API do *itjobs.pt*, permitindo listar ofertas de emprego, filtrar resultados por critérios específicos, e obter informações detalhadas sobre cada oferta.

Foi criada a função *get_api* para gerir a paginação da API e consolidar os resultados numa única estrutura de dados.

O módulo *re* foi utilizado para criar expressões regulares para limpeza de dados, extração de informações relevantes, e verificação de critérios específicos, como a pesquisa de salários em descrições textuais.

Dados estruturados foram organizados e exportados em formatos JSON e CSV para facilitar a sua interpretação e análise.

A biblioteca *typer* foi essencial para a criação de uma interface de linha de comandos (CLI), permitindo uma interação simples e eficiente com as funcionalidades desenvolvidas.

Para cumprir os objetivos do TP1, foi implementado o uso de módulo *requests* e *re*. A CLI implementada inclui comandos para:

- Listar trabalhos mais recentes, possibilitando a exportação para CSV.
- Listar os trabalhos do tipo *Full-Time*, de um determinado tipo, numa dada localização, possibilitando a exportação para CSV.
- Extrair e apresentar o salário de um trabalho específico, possibilitando a exportação para CSV.
- Obter o id de um trabalho que requerem uma determinada lista de *skills*, possibilitando a exportação para CSV.

Para complementar o que foi pedido para o primeiro trabalho, foram adicionados comandos para:

- Listar os trabalhos remotos, numa dada localização, para um determinado tipo de contrato.
- Criar um documento PDF com as localidades e, para cada uma, listar os trabalhos oferecidos e respetivas informações.

Para cumprir os objetivos do TP2, foi implementado um mecanismo de *web scraping* utilizando o módulo *BeautifulSoup*, que permitiu recolher informações adicionais sobre empresas no website *AmbitionBox*. À CLI foram adicionados comandos para:

- Recolher informações de um trabalho e respetiva empresa, com possibilidade de exportação para CSV.
- Criar um ficheiro CSV com uma contagem de vagas por cada trabalho para cada localização.
- Listar, para um dado trabalho, as principais 10 skills e respetivas ocorrências.

Para o website *Indeed*, foi implementada a funcionalidade de recolher informações de um trabalho e respetiva empresa, com possibilidade de exportação para CSV.

Para complementar o que foi pedido para o segundo trabalho, foram adicionadas funcionalidades para:

- Haver a possibilidade de criação de um ficheiro HTML com as informações recolhidas de um dado trabalho.
- Listar as localidades e as vagas disponíveis para um dado trabalho.

Importação dos Dados

Recolha e Integração de Dados da API

```
url = "https://api.itjobs.pt/job/list.json?api_key=09ad1042ebaf1704533805cd2fab64f1"
headers = {"User-Agent": "Mozilla/5.0"}
```

Figura 1 - Importação da API

Para importar a API do *itjobs.pt*, foi criada a função *get_api*.

A função realiza múltiplas requisições consecutivas à API, página por página, até que todos os resultados disponíveis sejam recolhidos, iniciando a recolha a partir da primeira página e, progressivamente, adicionar os resultados de cada página a uma lista. Quando todos os resultados são integrados na lista, a função finaliza e retorna a lista de resultados. Dessa forma, é possível obter o total de trabalhos da API.

Exportação de Dados

Função *export_to_csv1*

```
def export_to_csv1(data, export_csv: Optional[str])
```

Figura 2 - Função *export_to_csv1*

Esta função permite exportar um conjunto de dados estruturados para um ficheiro no formato CSV, tornando-os acessíveis e utilizáveis fora do contexto do programa. É criado um ficheiro CSV e ao mesmo é adicionado um cabeçalho com os nomes das colunas que descrevem os campos principais das ofertas de emprego. Os dados fornecidos são processados linha a linha, sendo cada entrada transformada e formatada adequadamente para preencher as colunas do ficheiro.

Função *export_to_csv2*

```
def export_to_csv2(data, export_csv: Optional[str])
```

Figura 3 - Função *export_to_csv2*

Esta função permite exportar dados estruturados para um ficheiro CSV, oferecendo flexibilidade para lidar com diferentes formatos de entrada. A função suporta dois tipos principais de dados: um único dicionário ou uma lista de dicionários. A função identifica automaticamente os campos disponíveis nos dados fornecidos e utiliza-os como cabeçalhos das colunas no ficheiro CSV. Em seguida, escreve os registos correspondentes às linhas do ficheiro, garantindo que a informação é bem organizada e consistente.

Tratamento de Dados

```
def limpar_html(texto_html):
    texto_limpo = re.sub(r"<br\s*/?>", " ", texto_html) # substitui <br> por espaço
    texto_limpo = re.sub(r"</?p>", " ", texto_limpo) # substitui <p> por espaço
    texto_limpo = re.sub(r"</?strong>", "", texto_limpo) # remove <strong> e </strong>
    texto_limpo = re.sub(r"</?a[^>]*>", "", texto_limpo) # remove tags <a> (links)
    texto_limpo = re.sub(r"<[^>]+>", "", texto_limpo) # remove qualquer outra tag HTML restante
    texto_limpo = re.sub(r"\n\s*\n", " ", texto_limpo).strip() # substitui múltiplas quebras de linha por
    espaço
    return texto_limpo
```

Figura 4 - Função *limpar_html*

Para que ao exportar dados para CSV, as informações obtidas em formato JSON estivessem visualmente mais compreensíveis, recorreu-se ao uso de expressões regulares para limpar o texto da descrição dos vários trabalhos da API. Para isso usou-se o módulo *re*, especialmente o *re.sub* de forma a substituir, neste caso, as *tags* presentes no texto, por espaços em branco, ou nulos.

Funcionalidades

Trabalho Prático 1

Função *listar_trabalhos*

```
@app.command()
def listar_trabalhos(
    n: int = typer.Argument(10, help="Número de trabalhos para listar"),
    export_csv: Optional[str] = typer.Option(None, help="Exporta os dados para um arquivo CSV")
)
```

Figura 5 - Função *listar_trabalhos*

Esta função permite fornecer ao utilizador uma lista dos trabalhos mais recentes disponíveis, com a opção de exportar os dados para um ficheiro CSV.

A função obtém a totalidade dos dados de trabalhos através de uma API e organiza estes trabalhos por ordem decrescente de data de publicação. De seguida, seleciona os *n* trabalhos mais recentes, conforme especificado pelo utilizador, e apresenta as informações no terminal em formato JSON. Caso o utilizador não defina o número de trabalhos a serem apresentados, por defeito, são apresentados os 10 mais recentes.

Exportar os dados para um ficheiro CSV é opcional, pelo que, se o utilizador optar por exportar os dados, a função chama a função dedicada a gravar os trabalhos seleccionados num ficheiro CSV.

Exemplo:

```
python projeto.py listar-trabalhos 1 --export-csv "trabalhos_recentes.csv"
{
  {
    "id": 494487,
    "company": {
      "id": 4598,
      "name": "KWAN",
      "logo": "https://static.itjobs.pt/images/companies/85/e55/4598/logo.png",
      "description": "Parte do grupo Rupeal, a Kwan é uma empresa de IT Staffing especializada na arte da realização profissional, assegurando que quem trabalha connosco, sejam clientes, candidatos ou consultores, está no caminho certo para alcançar as suas motivações!",
      "address": "Avenida Duque de Ávila, 46 - 3ªA\r\n1050 - 083 Lisboa",
      "url": "http://www.kwan.pt/",
      "url_twitter": "https://twitter.com/kwancommunity",
      "url_facebook": "https://www.facebook.com/KWANcommunity/",
      "url_linkedin": "https://www.linkedin.com/company/kwan-love-respect-community",
      "slug": "kwan"
    },
    "companyId": 4598,
    "title": "Senior Data Engineer",
    "body": "<p>At KWAN, we don't just offer jobs - we provide platforms for Building a Bright Future. We work with a wide range of technologies, and can align your unique skills, passions, and professional background to place you in a project that matters. Growth is very important in KWAN, and we will guide you throughout your career journey, with an entire team of dedicated People Managers.</p><p>Currently, we are seeking a Data Engineer who's eager to continuing to evolve, thrives on challenges, and is ready to embark in a new opportunity.</p><p><strong>Responsibilities:</strong></p><ul><li>Design and maintain robust data pipelines, ensuring seamless data flow from various sources to storage and analytical platforms;</li><li>Craft and optimize data models for efficient data retrieval and analysis, balancing performance and scalability;</li><li>Implement rigorous testing and validation protocols to guarantee the accuracy and integrity of data;</li><li>Proactively identify and solve data processing issues, enhancing the performance and reliability of data systems;</li><li>Engage in large-scale data projects, utilizing frameworks like Apache Spark for efficient data handling and processing;</li><li>Work on cutting-edge projects that process and analyze data in real-time, ensuring swift data delivery and insights.</li></ul><p></p><p><strong>Requirements:</strong></p><ul><li>At least 3 years of experience in data engineering;</li><li>Experience working with public cloud platforms, especially AWS.</li><li>Experience with IaC tools, such as Terraform.</li><li>Ability to code in at least one programming language (Python, Java, JavaScript, Scala) is a must.</li><li>Experience working with and creating data-driven architectures, preferably in public cloud platforms.</li><li>Experience using a querying language such as SQL.</li><li>Experience with Big Data and data modelling.</li><li>Advanced knowledge of multiple SQL/noSQL data environments and a strong understanding of why and when to use a particular solution.</li><li>Familiarity with tools in the Big Data ecosystem (like Hadoop, Cassandra, Apache Hive, Apache Spark, and others alike).</li><li>Experience with orchestration tools like Airflow is a big plus.</li></ul><p><strong>Offer:</strong></p><ul><li>Empowering and respectful work culture where every voice matters.</li><li>Personalized Coaching Program: Provided by in-house specialists, our People Managers</li><li>Continuous Learning: Join our tech meetups and online trainings.</li><li>Dynamic Community Culture: From movie and quizz nights to dinners and our annual summer gathering, we believe in having fun together.</li><li>Celebrating You: Through our Rocket Points Employee Rewards System, we constantly recognize your contributions.</li><li><strong>Seamless Relocation: For our international hires, we offer a comprehensive relocation package and visa sponsorship to ensure a smooth transition.</strong></li></ul><p></p><p>If this offer resonates with your career aspirations and you're ready to embrace an exciting new challenge, we encourage you to seize this opportunity <a href='\"https://bit.ly/3vVXjGa\"'>https://bit.ly/3vVXjGa</a></p><p></p><p>Not sure yet? No problem! Reach out anyway, and let our team guide you through the best opportunities we have for you.</p>,<br>
    "ref": "",
    "allowRemote": false,
    "wage": null,
    "types": [
      {
        "id": "1",
        "name": "Full-time"
      }
    ],
    "locations": [
      {
        "id": "18",
        "name": "Porto"
      }
    ],
    "publishedAt": "2024-12-30 22:01:11",
    "updatedAt": "2024-12-30 22:01:11",
    "slug": "senior-data-engineer"
  }
}
```

Figura 6 - Exemplo output 'listar-trabalhos'

Função `full_time_jobs`

```
@app.command()
def full_time_jobs(
    company_name: str = typer.Argument(..., help="Nome da empresa desejada"),
    location: str = typer.Argument(..., help="Localização da empresa"),
    limit: int = typer.Argument(1, help="Número máximo de resultados a exibir"),
    export_csv: Optional[str] = typer.Option(None, help="Exporta os dados para um arquivo CSV")
)
```

Figura 7 - Função `full_time_jobs`

Nesta função é dada a oportunidade ao utilizador de inserir o nome da empresa e uma localização, bem como o número de trabalhos correspondentes que este quer que sejam apresentados. Assim, através da verificação dos dados inseridos pelo utilizador, caso sejam correspondentes aos disponíveis na API, são apresentados os dados do trabalho do tipo 'Full-Time' em formato JSON para a empresa e localização especificadas e, caso solicitado, estes são exportados para um ficheiro CSV.

Nesta função dois dos parâmetros são opcionais. O 'limit', caso não seja inserido, é automaticamente definido como sendo 1 e, exportar o resultado obtido para CSV, pode ser também uma opção a ter em conta pelo utilizador.

Exemplo:

```
python projeto.py full-time-jobs "kwan" porto 1 --export-csv "trabalhos_full_time.csv"
[
  {
    "id": 494487,
    "title": "Senior Data Engineer",
    "company": {
      "id": 4598,
      "name": "KWAN",
      "logo": "https://static.itjobs.pt/images/companies/85/e55/4598/logo.png",
      "description": "Parte do grupo Rupeal, a Kwan é uma empresa de IT Staffing especializada na arte da realização profissional, assegurando que quem trabalha connosco, sejam clientes, candidatos ou consultores, está no caminho certo para alcançar as suas motivações!",
      "address": "Avenida Duque de Ávila, 46 - 3ªA\`n1050 - 083 Lisboa",
      "url": "http://www.kwan.pt/",
      "url_twitter": "https://twitter.com/kwancommunity",
      "url_facebook": "https://www.facebook.com/KWANcommunity/",
      "url_linkedin": "https://www.linkedin.com/company/kwan-love-respect-community",
      "slug": "kwan"
    },
    "body": "At KWAN, we don't just offer jobs - we provide platforms for Building a Bright Future. We work with a wide range of technologies, and can align your unique skills, passions, and professional background to place you in a project that matters. Growth is very important in KWAN, and we will guide you throughout your career journey, with an entire team of dedicated People Managers. Currently, we are seeking a Data Engineer who's eager to continuing to evolve, thrives on challenges, and is ready to embark in a new opportunity. Responsibilities: Design and maintain robust data pipelines, ensuring seamless data flow from various sources to storage and analytical platforms; Craft and optimize data models for efficient data retrieval and analysis, balancing performance and scalability; Implement rigorous testing and validation protocols to guarantee the accuracy and integrity of data; Proactively identify and solve data processing issues, enhancing the performance and reliability of data systems; Engage in large-scale data projects, utilizing frameworks like Apache Spark for efficient data handling and processing; Work on cutting-edge projects that process and analyze data in real-time, ensuring swift data delivery and insights. Requirements: At least 3 years of experience in data engineering; Experience working with public cloud platforms, especially AWS. Experience with IaC tools, such as Terraform. Ability to code in at least one programming language (Python, Java, JavaScript, Scala) is a must. Experience working with and creating data-driven architectures, preferably in public cloud platforms. Experience using a querying language such as SQL. Experience with Big Data and data modelling. Advanced knowledge of multiple SQL/noSQL data environments and a strong understanding of why and when to use a particular solution. Familiarity with tools in the Big Data ecosystem (like Hadoop, Cassandra, Apache Hive, Apache Spark, and others alike). Experience with orchestration tools like Airflow is a big plus. Offer: Empowering and respectful work culture where every voice matters. Personalized Coaching Program: Provided by in-house specialists, our People Managers. Continuous Learning: Join our tech meetups and online trainings. Dynamic Community Culture: From movie and quiz nights to dinners and our annual summer gathering, we believe in having fun together. Celebrating You: Through our Rocket Points Employee Rewards System, we constantly recognize your contributions. Seamless Relocation: For our international hires, we offer a comprehensive relocation package and visa sponsorship to ensure a smooth transition. If this offer resonates with your career aspirations and you're ready to embrace an exciting new challenge, we encourage you to seize this opportunity https://bit.ly/3vVXjGa Not sure yet? No problem! Reach out anyway, and let our team guide you through the best opportunities we have for you.",
    "publishedAt": "2024-12-30 22:01:11",
    "wage": null,
    "locations": [
      {
        "id": "18",
        "name": "Porto"
      }
    ]
  }
]
```

Resultados exportados para o arquivo CSV: trabalhos_full_time.csv

Figura 8 - Exemplo output 'full-time-jobs'

Função `remote_jobs`

```
@app.command()
def remote_jobs(
    location: str = typer.Argument(..., help="Localização da empresa"),
    contract: str = typer.Argument(..., help="Tipo de contrato desejado"),
)
```

Figura 9 - Função `remote_jobs`

De forma semelhante à função `full_time_jobs`, nesta função também é apresentado um resultado em formato JSON, conforme os dados introduzidos pelo utilizador. Neste caso, é possível escolher qual a localização pretendida e o tipo de contrato (*Full-Time*, *Permanent*, ...), sendo o output obtido, os trabalhos de cada empresa que aceitam trabalho remoto nas condições inseridas.

Exemplo:

```
python projeto.py remote-jobs braga permanent
Trabalho remoto encontrado: Senior Linux Systems Administrator na empresa Eurotux
Trabalho remoto encontrado: Senior DevOps AWS na empresa Eurotux
Trabalho remoto encontrado: Senior DevOps Azure na empresa Eurotux
Trabalho remoto encontrado: Frontend Developer - Angular na empresa Growin - Know to grow
Trabalho remoto encontrado: Backend Developer - Java na empresa Growin - Know to grow
Trabalho remoto encontrado: Senior Fullstack Developer (Node.JS / Angular) na empresa Altar.io
Trabalho remoto encontrado: Senior Back-End Developer na empresa Altar.io
Trabalho remoto encontrado: Lead Fullstack Developer (Node.JS / Angular) na empresa Altar.io
Trabalho remoto encontrado: SOC Manager na empresa Eurotux
Trabalho remoto encontrado: SharePoint Specialist na empresa Eurotux
Trabalho remoto encontrado: Senior Project Manager na empresa Altar.io
Trabalho remoto encontrado: Security Architect na empresa Adentis
Trabalho remoto encontrado: Data Engineer na empresa Altar.io
Trabalho remoto encontrado: Fullstack Developer (Node.JS / Angular) na empresa Altar.io
Trabalho remoto encontrado: Programador .NET Full Stack na empresa Make IT Dynamic
Trabalho remoto encontrado: Consultor Técnico de ERP (Cegid | Primavera) na empresa Make IT Dynamic
Trabalho remoto encontrado: DBA SQL Server e PostgreSQL na empresa B-Simple
```

Figura 10 - Exemplo output 'remote-jobs'

Função `get_salary`

```
@app.command()
def get_salary(job_id: int)
```

Figura 11 - Função `get_salary`

Esta função permite obter o salário associado a uma vaga específica identificada pelo seu id. Esta ferramenta é ideal para utilizadores que precisam de informações rápidas e específicas sobre salários em ofertas de emprego, mesmo quando os dados não estão diretamente disponíveis num campo estruturado.

A função faz uma requisição à API para obter os detalhes das ofertas de emprego, filtra a vaga desejada utilizando o id fornecido e tenta extrair o salário a partir de dois métodos: um campo específico chamado *wage* e pesquisa no texto do campo *body* usando expressões regulares.

Se o salário for encontrado, o programa exibe a informação; caso contrário, informa que o salário não foi localizado.

Exemplo:

```
python projeto.py get-salary 493195
Salário encontrado no campo específico: 35000
```

Figura 12 - Exemplo output 'get-salary'

Função `skills`

```
@app.command()
def skills(
    skills: str = typer.Argument(..., help="Lista de skills separadas por vírgulas, por exemplo:
python,django,sql"),
    export_csv: Optional[str] = typer.Option(None, help="Exporta os dados para um arquivo CSV")
)
```

Figura 13 - Função `skills`

A funcionalidade `skills` tem como objetivo filtrar ofertas de emprego com base numa lista específica de competências fornecida pelo utilizador e, opcionalmente, exportar os resultados para um ficheiro CSV. Após processar os dados, a funcionalidade filtra ofertas de emprego que correspondam a todas as competências especificadas, apresenta os resultados em formato JSON formatado e exporta os resultados para um ficheiro CSV, se solicitado.

Exemplo:

```
python projeto.py skills python,mysql --export-csv "trabalhos_com_skills_pedidas.csv"
Job ID: 493160
Job ID: 494378
Job ID: 494360
Job ID: 492788
Job ID: 492704
Job ID: 494034
Job ID: 494037
Job ID: 494038
Job ID: 494039
```

```

Job ID: 494035
Job ID: 494036
Job ID: 493948
Job ID: 492300
Job ID: 493788
Job ID: 493804
Job ID: 493648
Job ID: 492088
Job ID: 493669
Job ID: 491931
Job ID: 491928
Job ID: 493393
Job ID: 493392
Job ID: 493255
Job ID: 493391
Job ID: 493324
Job ID: 491687
Job ID: 493172
Job ID: 493122
Resultados exportados para o arquivo CSV: trabalhos_com_skills_pedidas.csv

```

Figura 14 - Exemplo output 'skills'

Função `trabalhos_por_localidade`

```

@app.command()
def trabalhos_por_localidade()

```

Figura 15 - Função `trabalhos_por_localidade`

A função organiza os dados numa estrutura hierárquica, associando cada localidade às empresas que têm ofertas nessa região. Dentro de cada empresa, é gerada uma lista detalhada dos trabalhos, incluindo informações como título, email de contacto e URL da empresa.

Após organizar os dados, a função exibe as informações no terminal de forma hierárquica, para facilitar a leitura, e cria um ficheiro PDF com o mesmo conteúdo, permitindo que os dados sejam armazenados ou partilhados para análise posterior.

Exemplo:

```

python projeto.py trabalhos-por-localidade
Dados exportados para 'trabalhos por localidade.pdf' com sucesso!

```

Figura 16 - Output 'trabalhos-por-localidade'

Trabalhos Organizados por Localidade	
Localização: Lisboa Empresa: Satcom Europe Trabalho: Legal & Compliance Assistant email: N/A url: https://td.satcom-int.com Empresa: KWAN Trabalho: Technical Support Engineer email: N/A url: http://www.kwan.pt/ Empresa: KCS IT Trabalho: React developer email: N/A url: http://www.kcsit.pt Empresa: Adentis Trabalho: Data Analyst email: geral@adentis.pt url: https://adentis.pt/ Empresa: Mind Source Trabalho: QA Lead email: recrutamento@mindsource.pt url: http://www.mindsource.pt Empresa: Landskill Trabalho: Systems Administrator email: N/A url: https://www.landskill.com Empresa: Integer Consulting Trabalho: Programador Angular email: accountpay@integer.pt url: http://www.integer.pt Localização: Porto Empresa: GrupoPIE, Portugal SA Trabalho: Programador Web PHP email: geral@grupopie.com url: http://www.grupopie.com Empresa: KCS IT	Trabalho: React developer email: N/A url: http://www.kcsit.pt Empresa: Landskill Trabalho: Systems Administrator email: N/A url: https://www.landskill.com Localização: Leiria Empresa: KCS IT Trabalho: React developer email: N/A url: http://www.kcsit.pt Localização: Coimbra Empresa: To-Be Portugal Trabalho: Programador júnior - .NET Framework - SQL Server email: N/A url: https://www.tobe.co.ao/ Localização: Aveiro Empresa: akapeople Trabalho: Governance Specialist email: info@akapeople.pt url: https://www.akapeople.pt Localização: Braga Empresa: siosLIFE Trabalho: Frontend Developer/Engineer email: tr@sioslife.com url: http://www.sioslife.com/ForYoungerSpirits

Figura 17 - Exemplo pdf 'trabalhos-por-localidade'

Trabalho Prático 2

Função *informations*

```
@app.command()
def informations(
    job_id: int,
    export_csv: Optional[str] = typer.Option(None, help="Exporta os dados para um arquivo CSV"),
    generate_html: bool = typer.Option(False, help="Cria e abre o HTML com as informações.")
)
```

Figura 18 - Função *informations*

Esta função tem como objetivo recolher, combinar e apresentar informações detalhadas sobre uma oferta de trabalho específica, enriquecendo os dados provenientes da API do *ijobs* com informações adicionais recolhidas do website *AmbitionBox*.

A função recebe o id do trabalho escolhido pelo utilizador e recorre às funções *get_api_content* e *get_html_content* para obter os dados pretendidos.

```
def get_api_content(job_id)
Figura 19 - Função get_api_content
```

Esta função recebe o id do trabalho escolhido pelo utilizador e cria um dicionário com o id, respetivo título do trabalho e empresa que o oferece.

```
company_name1 = company_name.lower()
company_name2 = re.sub(r'\s+', '-', company_name1)
url = f"https://www.ambitionbox.com/overview/{company_name2}-overview"
headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:98.0) Gecko/20100101 Firefox/98.0",
           "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8",
           "Accept-Language": "en-US,en;q=0.5",
           "Accept-Encoding": "gzip, deflate",
           "Connection": "keep-alive",
           "Upgrade-Insecure-Requests": "1",
           "Sec-Fetch-Dest": "document",
           "Sec-Fetch-Mode": "navigate",
           "Sec-Fetch-Site": "none",
           "Sec-Fetch-User": "?1",
           "Cache-Control": "max-age=0"}
```

Figura 20 - Extração de dados HTML

Para complementar as informações obtidas da API para a empresa do trabalho escolhido pelo utilizador, foi feita uma função para poder extrair os dados do HTML do website. Dessa forma, foi necessário obter o nome da empresa através da API e transformá-lo para letras minúsculas e, através de uma expressão regular, substituir os espaços por '-', obtendo o url correspondente à empresa no site da *AmbitionBox*.

```
def get_html_content(soup)
Figura 21 - Função get_html_content
```

Tendo um *soup* correspondente ao url obtido, esta função permite obter o rating geral da empresa, a sua descrição e os principais benefícios de trabalhar nessa empresa.

Após obter todos os dados, se o utilizador assim o solicitar, estes podem ser exportados para um ficheiro CSV. Adicionalmente, pode também visualizar um ficheiro HTML na web com todas as informações obtidas e ainda com os contactos da empresa, se assim o desejar.

Exemplo:

```
python projeto2.py informations 494279 --export-csv "informations.csv" --generate-html
{
  "id": 494279,
  "title": "DevOps Manager",
  "company": "Qualitest",
  "overall_rating": "3.7",
```

```

"description": "Qualitest brings in the culture of AI-powered quality orchestration across your business, technology and operational landscape through our engineering-led process discipline. Qualitest is the world's leading managed services provider of AI-led quality engineering solutions. It helps brands transition through the digital assurance journey and make the move from conventional functional testing to adopt innovations such as automation, AI, blockchain, and XR. Qualitest's core mission is to mitigate business risks associated with digital adoption. It fulfills this through customized quality engineering solutions that leverage Qualitest's deep, industry-specific knowledge for various sectors, including technology, telecommunications, finance, healthcare, media, utilities, retail, manufacturing, and defense. These scalable solutions protect brands through end-to-end value demonstration with a focus on customer experience and release velocity. Qualitest has offices in the United States, United Kingdom, Germany, Israel, Romania, India, Mexico, Portugal, Switzerland, and Argentina. It employs more than 7,000 engineers who serve over 400 customers worldwide. A pioneer and innovator in its industry, Qualitest is the only services provider positioned by Everest Group as a Leader in both Next-generation Quality Engineering (QE) Services PEAK Matrix's Assessment 2023 and the Quality Engineering (QE) Specialist Services PEAK Matrix's Assessment 2023.",
"benefits": "work_policy: Hybrid, Monday to Friday, Flexible timing, No travel; top_employees_benefits: Job/Soft skill training, Health insurance, Office cab/shuttle, Free meal"
}
Dados exportados para o arquivo CSV: informations.csv
Página HTML criada: job_information.html

```

Figura 22 - Exemplo output 'informations'

Função *statistics*

```

@app.command()
def statistics(job_title: str = typer.Argument(None, help="Mostrar as vagas de um tipo de trabalho"))

```

Figura 23 - Função *statistics*

Esta função permite contar o número de vagas por tipo de trabalho em cada zona.

Os dados são organizados por localidade e a contagem de vagas é feita de forma estruturada, criando uma relação entre zonas, tipos de trabalho e o número de vagas disponíveis. Após gerar estas estatísticas, a função exporta os resultados completos para um ficheiro CSV, permitindo uma análise mais ampla e utilização em ferramentas externas.

Caso o utilizador especifique um título de trabalho, os resultados são filtrados e apresentados diretamente no terminal, destacando apenas as vagas relacionadas com o título fornecido.

Exemplo:

```

python projeto2.py statistics "python developer"
Ficheiro de exportação criado com sucesso.
Vagas para o trabalho 'python developer':
Zona: Lisboa, Tipo de Trabalho: Python Developer, Nº de Vagas: 3
Zona: Lisboa, Tipo de Trabalho: Python developer, Nº de Vagas: 1
Zona: Porto, Tipo de Trabalho: Python developer, Nº de Vagas: 1

```

Figura 24 - Exemplo output 'statistics'

Função *list_skills*

```

@app.command()
def list_skills(job_title: str, export_csv: Optional[str] = None)

```

Figura 25 - Função *list_skills*

Esta função tem como objetivo identificar e apresentar as *skills* mais frequentemente requeridas para um tipo específico de trabalho. Recorre a outras duas funções para obter os resultados: *get_jobs_urls* e *get_skills_from_job*.

```

job_title_formatted = re.sub(r'\s+', '-', job_title.lower())
url = f"https://www.ambitionbox.com/jobs/{job_title_formatted}-jobs-prf"
headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:98.0) Gecko/20100101 Firefox/98.0",
           "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8",
           "Accept-Language": "en-US,en;q=0.5",
           "Accept-Encoding": "gzip, deflate",
           "Connection": "keep-alive",
           "Upgrade-Insecure-Requests": "1",
           "Sec-Fetch-Dest": "document",
           "Sec-Fetch-Mode": "navigate",
           "Sec-Fetch-Site": "none",
           "Sec-Fetch-User": "?1",
           "Cache-Control": "max-age=0"}

```

Figura 26 - Extração de dados HTML

Para obter as *skills* desse trabalho, foi necessário obter o url com todas as ofertas de trabalho disponíveis para o mesmo. E, de seguida, obter o url de cada oferta de trabalho disponível, para obter uma lista com as 10 *skills* principais e, para cada uma delas, a contagem de ocorrências para o trabalho escolhido.

Exemplo:

```

python projeto2.py list-skills "data analyst" --export-csv "list_10skills.csv"
Total de URLs: 664
[
  {
    "skill": "sql",

```

```

    "count": 300
  },
  {
    "skill": "data analytics",
    "count": 249
  },
  {
    "skill": "python",
    "count": 224
  },
  {
    "skill": "data analysis",
    "count": 202
  },
  {
    "skill": "power bi",
    "count": 123
  },
  {
    "skill": "tableau",
    "count": 82
  },
  {
    "skill": "analytical chemistry",
    "count": 75
  },
  {
    "skill": "data visualization",
    "count": 70
  },
  {
    "skill": "clinical data management",
    "count": 58
  },
  {
    "skill": "machine learning",
    "count": 47
  }
]

```

Dados exportados para o arquivo CSV: list_skills.csv

Figura 27 - Exemplo output 'list-skills'

Função *informations2*

```

@app.command()
def informations2(
    job_id: int,
    export_csv: Optional[str] = typer.Option(None, help="Exporta os dados para um arquivo CSV"),
)

```

Figura 28 - Função *informations2*

Esta função tem como objetivo recolher, combinar e apresentar informações detalhadas sobre uma oferta de trabalho específica, enriquecendo os dados provenientes da API do *itjobs* com informações adicionais recolhidas do website *Indeed*.

O objetivo desta função é o mesmo que o da função *informations*, só que em vez de utilizarmos o website da *AmbitionBox*, utilizamos o *Indeed*.

Exemplo:

```

python projeto2.py informations2 494279 --export-csv "informations2.csv"
Qualitest
Website não encontrado para a empresa: Qualitest.
{
  "id": 494279,
  "title": "DevOps Manager",
  "company": "Qualitest",
  "overall_rating": "N/A",
  "description": "Informa\u00e7\u00f5es indispon\u00edveis."
}

```

Dados exportados para o arquivo CSV: informations2.csv

Figura 29 - Exemplo output 'informations2'

Ao analisarmos os termos e condições do website, constatamos que não permite *web scraping*. Os melhores websites que encontramos que poderiam complementar as informações da API seriam o *Indeed* e o *GlassDoor*. No entanto, como referido, o primeiro não permite *web scraping*, a não ser que seja feito um pedido por escrito, e o segundo necessita de registo. Pelo que criámos o código como se o website da *Indeed* permitisse *web scraping*.

Conclusão

O desenvolvimento deste projeto permitiu consolidar conhecimentos teóricos e práticos em áreas fundamentais da Ciência de Dados, como processamento de dados, integração de APIs e *Web Scraping*. Através da implementação de uma CLI com o auxílio da biblioteca *Typer*, foi possível criar uma solução eficiente e versátil para a extração, análise e enriquecimento de dados provenientes de diferentes fontes.

O uso de Expressões Regulares foi crucial para a manipulação de textos, enquanto as bibliotecas *requests* e *BeautifulSoup* facilitaram o enriquecimento de dados de websites. A exportação para JSON e CSV destacou o foco em ferramentas práticas e aplicáveis em contextos reais.

De forma geral, o projeto não apenas atingiu os objetivos propostos nos enunciados, mas também proporcionou uma oportunidade para explorar abordagens criativas e resolver problemas complexos, contribuindo significativamente para o desenvolvimento das competências técnicas na área de Ciência de Dados.